ASSIGNMENT-11.2

R.TEENESWARI

2303A51932

BATCH-30

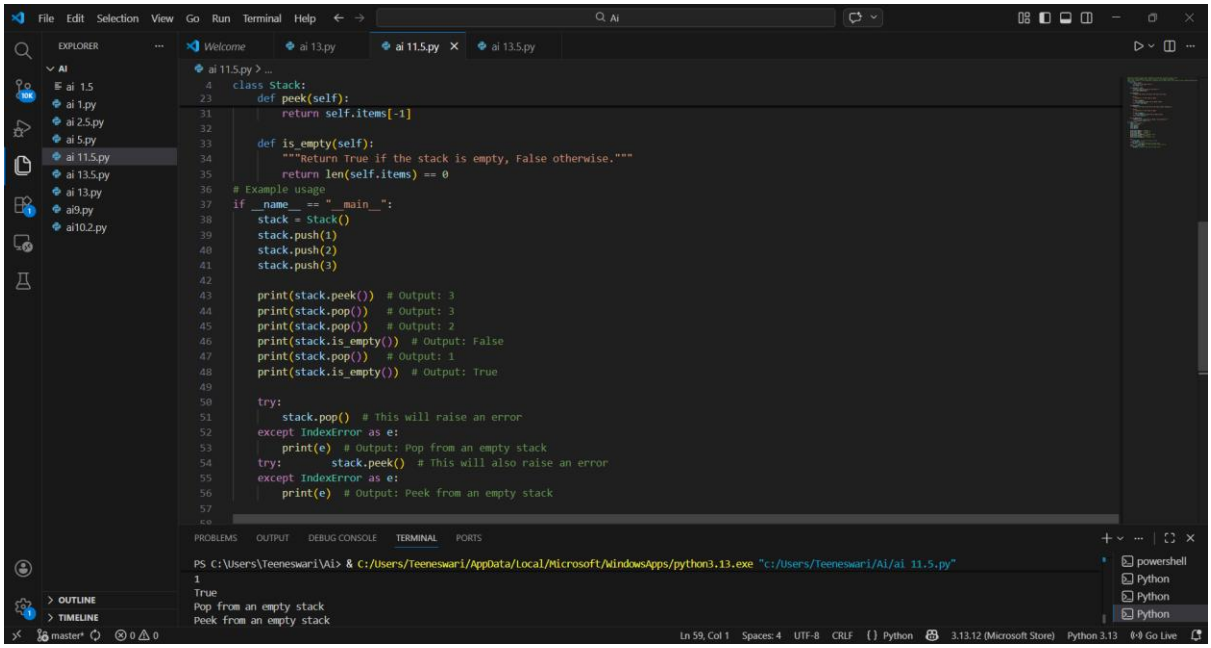
TASK-1:

PROMPT:

#Generate a python code to design and implement a Stack datastructure supporting basic stack operations.

A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

CODE:



EXPECTED OUTPUT:

A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

OBSERVATION :

-It is observed that in the out put will get by implementing FIFO principles

-The program successfully demonstrated stack and handled empty stcak condiitons

TASK-2:

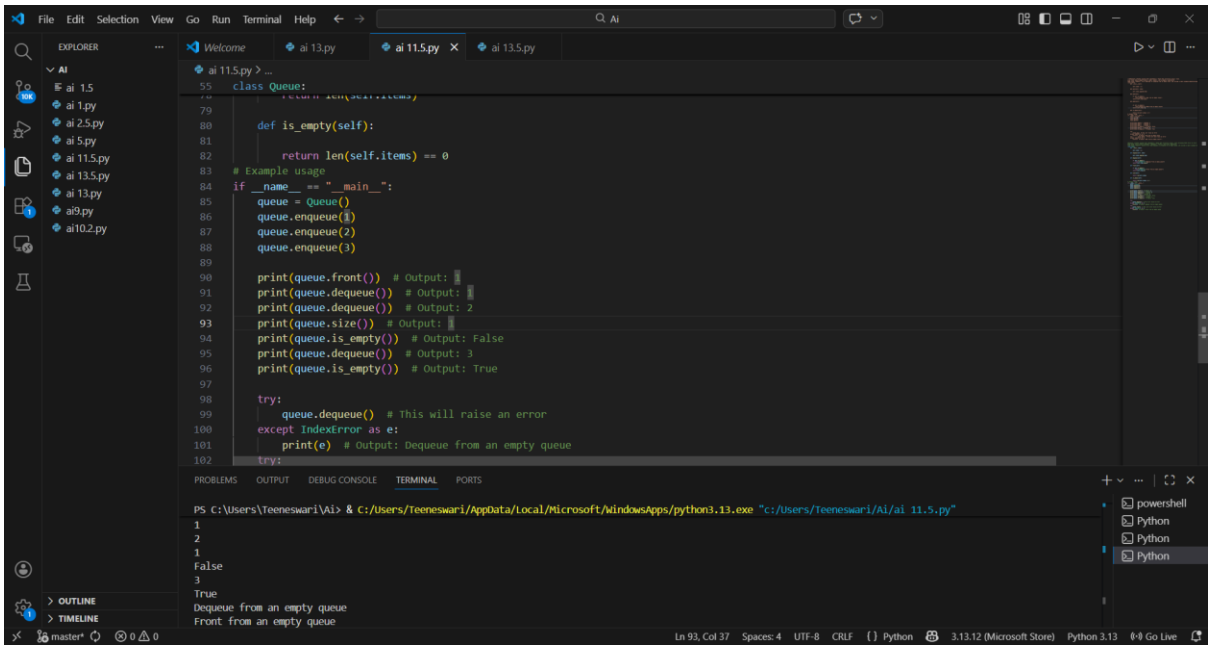
PROMPT:

#Generate a python code to create a Queue data structurefollowing FIFO principles

A complete Queue implementation including enqueue, dequeue,

front element access, and size calculation

CODE:



Expected Output:

• A complete Queue implementation including enqueue, dequeue,

front element access, and size calculation

OBSERVATION:

-it was able to implement the queue and dequeue ,font,size

- The program successfully demonstrated the queue operations

TASK-3:

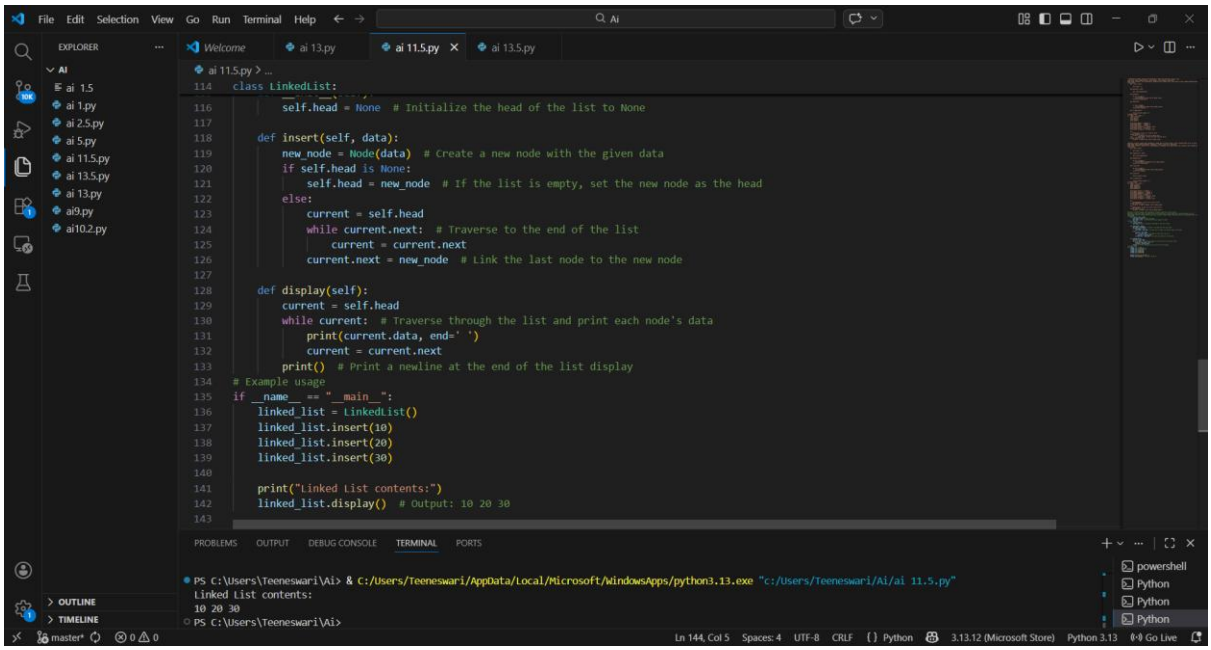
PROMPT: Generate a python code build a singly linked list supporting insertion

and traversal.

Correctly functioning linked list with node creation, insertion

logic, and display functionality.

CODE:



```
class LinkedList:
        self.head = None  # Initialize the head of the list to None

    def insert(self, data):
        new_node = Node(data)  # Create a new node with the given data
        if self.head is None:
            self.head = new_node  # If the list is empty, set the new node as the head
        else:
            current = self.head
            while current.next:  # Traverse to the end of the list
                current = current.next
            current.next = new_node  # Link the last node to the new node

    def display(self):
        current = self.head
        while current:  # Traverse through the list and print each node's data
            print(current.data, end=' ')
            current = current.next
        print()  # Print a newline at the end of the list display
# Example usage
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)

    print("Linked List contents:")
    linked_list.display()  # Output: 10 20 30
```

```
PS C:\Users\Teeneswari\Ai> & C:/Users/Teeneswari/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Teeneswari/Ai/ai 11.5.py"
Linked List contents:
10 20 30
PS C:\Users\Teeneswari\Ai>
```

Expected Output:

• Correctly functioning linked list with node creation, insertion

logic, and display functionality.

OBSERVATION:

-it was able to perform all the linked list operations

-it was getting the accurate expected output by the generated example usage code.

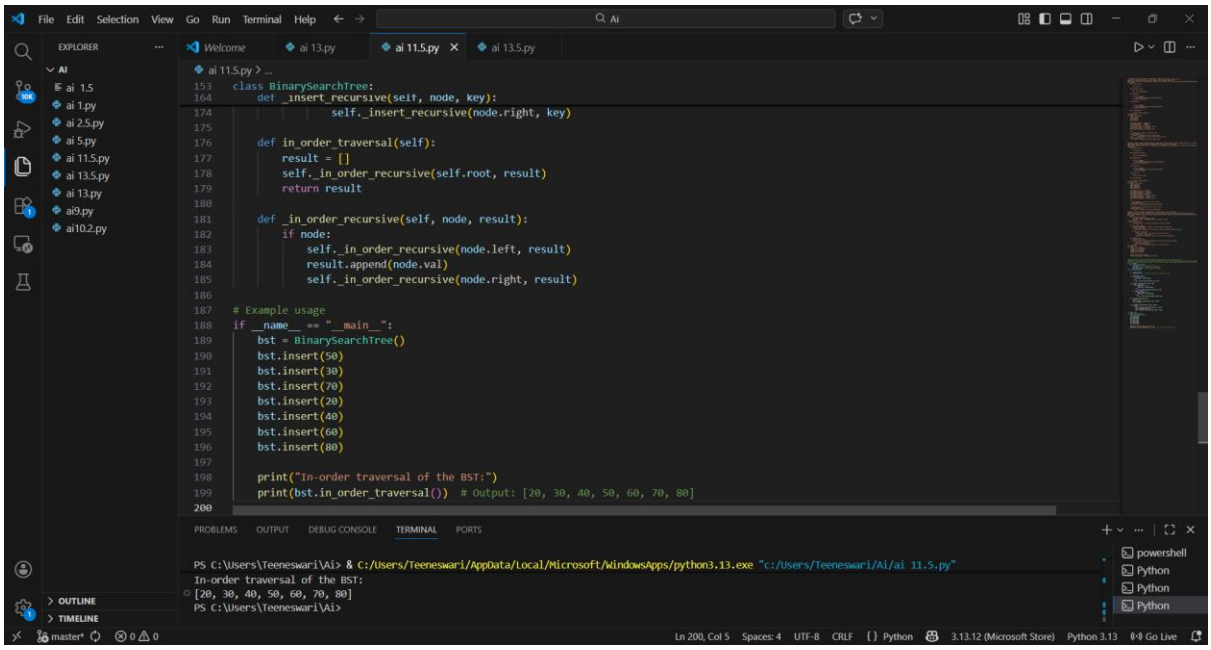
TASK-4:

PROMPT: Generate a python code to get a Binary Search Tree with AI support focusing on insertion and traversal.

#BST program with correct node insertion and in-order traversal

output.

CODE:



Expected Output:

• BST program with correct node insertion and in-order traversal

Output

OBSERVATION:

-it was able to perform all the BinarySearch Tree operations

-it was getting the accurate expected output by the generated example usage code.

TASK-5

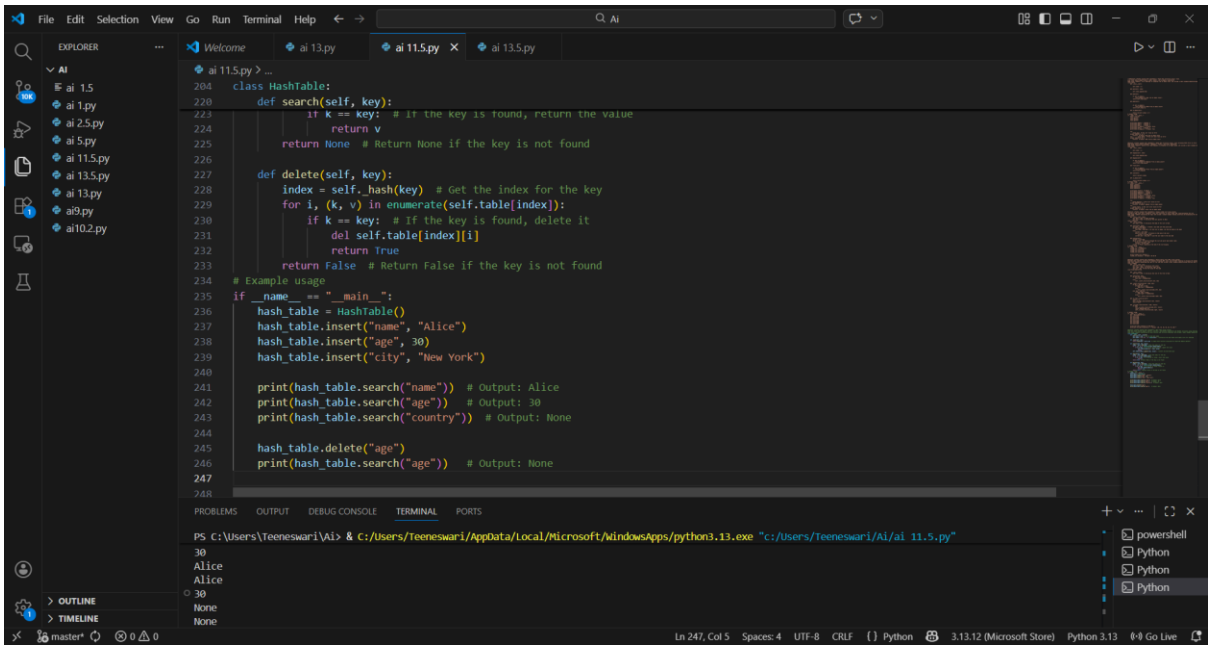
PROMPT:

Generate a python code to Create a hash table using AI with collision handling

#Hash table supporting insert, search, and delete using chaining

or open

CODE:



Expected Output:

• Hash table supporting insert, search, and delete using chaining

or open

OBSERVATION:

-it was able to perform all the Hash Table operations

-it was getting the accurate expected output by the generated example usage code.